

Planazo: informe tecnico del estado del proyecto

Fecha del analisis: 19 de agosto de 2026

Proyecto: planazo3

Tecnologia principal: Flutter/Dart con dependencias de Firebase

1. Resumen ejecutivo

Planazo es un prototipo de aplicacion movil para encontrar y organizar actividades deportivas. La version actual permite recorrer una experiencia visual completa: acceder desde una pantalla de login, consultar tres juntadas de ejemplo, abrir su detalle, crear una nueva juntada durante la sesion y consultar un perfil. La navegacion funciona con rutas imperativas de Flutter y el estado vive en memoria dentro de HomeScreen.

El proyecto esta en una **etapa de prototipo funcional de interfaz**, no en una beta lista para usuarios. Firebase aparece declarado en `pubspec.yaml`, existe configuracion de proyecto para Android y `firebase.json`, y se llama a `Firebase.initializeApp()`. Sin embargo, `lib/firebase_options.dart` esta vacio y no hay llamadas a Firebase Authentication ni Cloud Firestore. Por tanto, las credenciales, reuniones, perfil, participaciones y acciones de usuario todavia no son persistentes ni reales.

2. Alcance analizado

Se revisaron las fuentes Dart de `planazo3/lib`, el modelo duplicado en la raiz `models/meetup.dart`, `pubspec.yaml`, `analysis_options.yaml`, `README.md`, `firebase.json`, la configuracion Android de Firebase y la configuracion de lanzamiento de VS Code. Se excluyeron artefactos generados de `build/` y `.dart_tool/`.

No se encontraron archivos `*_test.dart`. En el entorno de analisis no estaban disponibles los ejecutables flutter ni git, por lo que el informe no afirma que flutter analyze, flutter test o una compilacion hayan sido ejecutados con exito.

3. Arquitectura actual

La arquitectura es una estructura Flutter pequena y directa, sin capas de dominio, repositorios ni gestion de estado externa.

- **Entrada y tema:** `lib/main.dart` inicializa Flutter, intenta inicializar Firebase y monta `MaterialApp` con tema Material 3 y color semilla naranja oscuro.
- **Autenticacion visual:** `lib/screens/login_screen.dart` muestra email y contrasena, pero el boton navega directamente a Home sin validar campos ni autenticar.
- **Listado y estado:** `lib/screens/home_screen.dart` mantiene una lista mutable de Meetup con tres datos de ejemplo. La lista se modifica localmente al volver de Create.
- **Presentacion reutilizable:** `lib/widgets/meetup_card.dart` representa cada actividad y selecciona un icono segun el deporte.
- **Creacion:** `lib/screens/create_screen.dart` recoge texto y numeros, construye un Meetup y lo devuelve a Home.
- **Detalle:** `lib/screens/detail_screen.dart` muestra la actividad y el boton Unirme solo presenta un `SnackBar`.
- **Perfil:** `lib/screens/profile_screen.dart` contiene datos y metricas fijas, y cerrar sesion solo vuelve a la pantalla anterior.

- **Modelo:** `lib/models/meetup.dart` es una clase mutable con siete campos obligatorios. No tiene identificador, serializacion, propietario, fecha tipada ni metodos `fromMap/toMap`.

Flujo principal:

`main.dart` -> `LoginScreen` -> `HomeScreen` -> `MeetupCard` -> `DetailScreen`

`HomeScreen` -> `CreateScreen` -> nuevo Meetup en memoria

`HomeScreen` -> `ProfileScreen`

4. Funcionamiento por pantalla

Inicio y login

`main()` asegura la vinculacion con el motor Flutter, espera `Firebase.initializeApp()` y luego inicia `SportMeetApp`. La pantalla inicial siempre es `LoginScreen`.

La pantalla contiene campos de email y contrasena, una identidad visual de Planazo y acciones Ingresar y Crear cuenta. Los controladores se crean dentro de `build`, no se liberan y sus valores no se utilizan. Ingresar ejecuta `pushReplacement` a `Home` independientemente de los datos introducidos. Crear cuenta no tiene comportamiento.

Home

`Home` muestra tres actividades precargadas: Basket en Parque Centenario, Futbol en Palermo y Running en Puerto Madero. Cada tarjeta informa deporte, lugar, fecha, hora y ocupacion. El boton de perfil abre `Profile` y el boton flotante abre `Create`.

Cuando `Create` devuelve un Meetup, `Home` lo agrega a la lista y llama a `setState`. Esto prueba el flujo de alta dentro de la sesion, pero al cerrar la app se pierde y no existe edicion, borrado, busqueda, filtros, ordenamiento ni control de duplicados.

Creacion de juntas

`Create` ofrece campos de deporte, lugar, fecha, hora, participantes actuales, maximo y descripcion. Los dos contadores empiezan con 1 y 10. Al pulsar Crear junta se intenta convertir los contadores con `int.tryParse`; ante entradas invalidas se usan silenciosamente 1 y 10. No hay campos obligatorios, validacion de rangos, selector de fecha/hora, prevencion de maximo menor que actuales ni dispose de controladores.

Detalle y union

`Detail` presenta icono, datos de ubicacion y agenda, ocupacion y descripcion. Unirse no cambia participantes ni guarda una relacion usuario-actividad; solamente muestra Te uniste a la actividad!. No existe estado de boton, cupo lleno, cancelar participacion ni confirmacion persistente.

Perfil

`Profile` muestra el nombre Maximiliano, una descripcion y metricas fijas: deportes favoritos, tres juntas creadas, 18 participaciones y calificacion 4.9. No hay datos derivados del usuario autenticado ni edicion de perfil. Cerrar sesion hace `Navigator.pop`, por lo que no limpia sesion ni devuelve necesariamente al login si la pantalla se abrio desde otro contexto.

5. Datos e integraciones

Modelo Meetup

El modelo representa una actividad, pero todos los valores temporales son `String` y no hay `id`. Esto impide ordenar de forma fiable, validar fechas, referenciar documentos de backend o identificar al creador. La clase tambien es mutable, de modo que cualquier pantalla puede modificar sus campos sin una regla clara.

Existe una copia del modelo en `models/meetup.dart` fuera de `planazo3/lib`; la aplicación importa la copia dentro de `lib/models`. La duplicación puede provocar cambios divergentes y debe eliminarse o convertirse en una dependencia explícita.

Firestore

Las dependencias declaradas son `firebase_core`, `cloud_firestore` y `firebase_auth`. `firebase.json` y `android/app/google-services.json` apuntan al proyecto Firebase `planazo-67`. No obstante, `lib/firebase_options.dart` está vacío, aunque `main.dart` inicializa Firebase sin pasar opciones. La configuración multiplataforma generada no está disponible en Dart.

No se observan lecturas o escrituras de Firestore, listeners en tiempo real, registro, login, recuperación de contraseña, cierre de sesión mediante `FirebaseAuth` ni reglas de seguridad documentadas. La integración backend está en estado de preparación, no de uso funcional.

6. Estado actual por capacidad

- **Interfaz visual:** prototipo navegable implementado.
- **Navegación:** login -> home, detalle, creación y perfil implementados localmente.
- **Listado de actividades:** solo datos semilla en memoria.
- **Alta de actividad:** funcional durante la sesión, sin validación robusta.
- **Detalle:** lectura visual implementada.
- **Unirse:** simulado con mensaje, sin persistencia.
- **Autenticación:** pantalla simulada; no funcional.
- **Perfil:** maqueta con valores hardcodeados.
- **Persistencia:** no implementada.
- **Firestore:** dependencias y parte de configuración presentes; código de opciones Dart y uso de servicios pendientes.
- **Calidad automatizada:** sin pruebas detectadas; no verificada por CLI en este entorno.
- **Documentación:** README aún es la plantilla inicial de Flutter.

La clasificación global es **MVP de interfaz / prototipo técnico temprano**. La app demuestra la idea y los flujos visuales principales, pero aún no constituye un producto multiusuario.

7. Riesgos y deuda técnica prioritaria

1. **La inicialización de Firebase puede fallar fuera de un entorno configurado**, porque `firebase_options.dart` está vacío y `initializeApp()` se llama sin opciones explícitas.
2. **El login no protege ninguna ruta**. Cualquier texto, incluso vacío, da acceso a Home.
3. **Los datos se pierden al reiniciar**. Esto afecta actividades, participaciones y cualquier futura cuenta si se construye sobre el estado actual.
4. **La entrada de usuario no está validada**. Hay riesgo de actividades incompletas, contadores inválidos y estados imposibles.
5. **No hay identificadores ni esquema persistente**. Integrar Firestore directamente con el modelo actual generaría documentos difíciles de actualizar y relacionar.
6. **No hay pruebas automatizadas**. Los flujos clave pueden romperse sin detección temprana.
7. **Los controladores no se liberan**. Las pantallas de login y creación deberían disponerlos en `dispose`.

8. La configuracion contiene identificadores y claves de Firebase en archivos del proyecto. Aunque las claves de cliente no son secretos por si mismas, el acceso debe protegerse con reglas Firebase y la configuracion no debe sustituir a controles de seguridad.

9. El README no describe el producto ni el arranque real. Falta documentar configuracion, plataformas soportadas y decisiones de datos.

8. Trabajo futuro recomendado

Fase 1: hacer ejecutable y estable el MVP

- Regenerar `firebase_options.dart` con FlutterFire CLI para las plataformas objetivo y verificar la inicializacion.
- Definir un entorno de ejecucion reproducible: version Flutter/Dart, comandos de instalacion, emulador/dispositivo y configuracion Firebase.
- Agregar `dispose` a controladores, Form con validadores y selectores tipados de fecha y hora.
- Convertir Meetup en un modelo persistente con `id`, `ownerId`, `DateTime`, `createdAt`, `fromMap` y `toMap`.
- Crear pruebas de widget para login, listado, alta, detalle y boton Unirme.

Fase 2: autenticacion y persistencia

- Implementar registro, inicio de sesion, cierre de sesion y recuperacion de contrasena con Firebase Auth.
- Crear una capa `MeetupRepository` para Firestore, con estados de carga, error y vacio.
- Guardar actividades en una coleccion con propietario, capacidad, fecha, ubicacion y descripcion.
- Persistir participaciones de forma idempotente, impedir unirse dos veces y controlar cupos.
- Añadir reglas Firestore basadas en usuario autenticado, propietario y operaciones permitidas.

Fase 3: producto utilizable

- Sustituir el perfil fijo por datos reales y permitir editar nombre, descripcion y deportes.
- Añadir busqueda, filtros por deporte/fecha/lugar, ordenamiento y refresco.
- Permitir editar y eliminar actividades propias.
- Añadir estados de actividad, cancelacion, notificaciones y manejo de conflictos de cupo.
- Mejorar accesibilidad, internacionalizacion, mensajes de error y estados offline.

Fase 4: calidad y lanzamiento

- Incorporar pruebas unitarias, de widget e integracion para Auth, Firestore y reglas de seguridad.
- Configurar CI para `flutter analyze`, `flutter test` y builds por plataforma.
- Separar configuraciones de desarrollo, staging y produccion.
- Revisar privacidad, consentimiento, moderacion de contenido y proteccion de datos personales.
- Generar builds firmadas y completar la documentacion de despliegue en tiendas.

9. Criterio de listo para una beta

Planazo deberia considerarse beta cuando un usuario pueda crear una cuenta, iniciar y cerrar sesion, crear una actividad valida, verla despues de reiniciar la app, unirse una sola vez si hay cupo, cancelar su participacion,

consultar un perfil real y recibir errores claros. Ese flujo debe estar cubierto por pruebas automaticas y protegido por reglas de Firestore verificadas en un proyecto de staging.

10. Conclusion

El proyecto tiene una base visual clara y suficiente para validar la idea de una red de encuentros deportivos. La siguiente inversion no deberia centrarse en mas pantallas decorativas, sino en convertir los flujos simulados en contratos reales: identidad, modelo de datos, persistencia, validacion y pruebas. Una vez resueltos esos fundamentos, las funciones sociales y de descubrimiento podran crecer sin rehacer la estructura principal.